

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

“Single Linked List”

**disusun Oleh:
Ardra Athailla Firsy
2511533028**

**Dosen Pengampu:
Dr. WAHYUDI, S.T, M.T**

**Asisten Praktikum:
Sherly Sukmadira**



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum *Struktur Data* ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data. Dalam laporan ini, penulis memaparkan hasil praktikum yang telah dilakukan, mulai dari konsep dasar hingga implementasi struktur data seperti array, stack, queue, dan linked list dalam pemrograman.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang.

Tidak lupa, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu dalam pelaksanaan praktikum serta penyusunan laporan ini.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep dan penerapan struktur data.

Padang, 5 Mei 2025

Ardra Athailla Firsy

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	4
1.1 Latar Belakang.....	4
1.2 Tujuan.....	4
1.3 Manfaat.....	4
BAB II PEMBAHASAN.....	6
2.1 Pengertian Single Linked List	6
2.2 Implementasi Stack pada Java.....	6
2.2.1 Class Node SLL (NodeSLL_2511533028)	7
2.2.2 Class Pencarian SLL (PencarianSLL_2511533028).....	8
2.2.3 Class Tambah SLL (TambahSLL_2511533028)	9
2.2.4 Class Reverse Date (ReverseDate_2511533028).....	11
BAB III PENUTUP	14
3.1 Kesimpulan.....	14
3.2 Saran	14
DAFTAR PUSTAKA.....	16
LAPORAN TUGAS PRAKTIKUM	17
1. Class Pasien (Pasien_2511533028)	17
2. Class Antrian Rumah Sakit (RumahSakit_2511533028)	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengolahan data menjadi hal yang sangat penting, terutama ketika berhubungan dengan struktur data yang dinamis. Salah satu struktur data yang sering digunakan adalah **Single Linked List**. Struktur ini merupakan kumpulan elemen (node) yang saling terhubung melalui pointer atau referensi, di mana setiap node hanya menunjuk ke satu node berikutnya.

Single Linked List menjadi alternatif dari array karena memiliki keunggulan dalam pengelolaan memori yang lebih fleksibel. Jika pada array ukuran harus ditentukan di awal, maka pada linked list ukuran dapat bertambah atau berkurang secara dinamis sesuai kebutuhan. Hal ini sangat berguna dalam aplikasi yang membutuhkan manipulasi data secara sering, seperti penambahan dan penghapusan elemen.

Selain itu, pemahaman tentang Single Linked List menjadi dasar penting untuk mempelajari struktur data lain yang lebih kompleks, seperti double linked list, stack, queue, hingga graph. Oleh karena itu, materi ini penting untuk dipelajari oleh mahasiswa atau programmer pemula.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 5 tentang Single Linked List adalah sebagai berikut:

1. Memahami konsep dasar struktur data Single Linked List
2. Mengetahui cara kerja node dan pointer dalam menghubungkan data.
3. Mampu mengimplementasikan Single Linked List dalam pemrograman.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 5 tentang Single Linked List adalah sebagai berikut:

1. Efisiensi Memori.
2. Kemudahan Manipulasi Data.
3. Dasar Pemahaman Struktur Data Lanjutan
4. Meningkatkan Logika Pemrograman

BAB II PEMBAHASAN

2.1 Pengertian Single Linked List

Single Linked List adalah struktur data linear yang terdiri dari sekumpulan elemen yang disebut node, di mana setiap node saling terhubung secara berurutan melalui sebuah pointer. Setiap node memiliki dua bagian utama, yaitu data yang berfungsi untuk menyimpan nilai atau informasi, serta pointer (next) yang berfungsi untuk menyimpan alamat atau referensi ke node berikutnya. Disebut *single* karena setiap node hanya memiliki satu pointer yang mengarah ke node selanjutnya, sehingga hubungan antar node bersifat satu arah (forward).

Struktur ini diawali dari node pertama yang disebut **head**, dan untuk mengakses data harus dilakukan secara berurutan dengan mengikuti pointer dari satu node ke node berikutnya hingga mencapai node terakhir yang menunjuk ke **NULL**, yang menandakan akhir dari rangkaian data. Berbeda dengan array, Single Linked List tidak disimpan dalam memori yang berurutan, sehingga bersifat dinamis dan memungkinkan penambahan maupun penghapusan data dengan lebih fleksibel tanpa harus menentukan ukuran di awal.

Dengan karakteristik tersebut, Single Linked List sangat cocok digunakan dalam kondisi yang membutuhkan manipulasi data secara dinamis, meskipun memiliki keterbatasan dalam hal akses langsung terhadap elemen tertentu karena harus dilakukan secara traversal dari node pertama.

2.2 Implementasi Stack pada Java

Pada praktikum pekan 5 mahasiswa disuruh untuk mengikuti intruksi dari dosen. Yaitu menuliskan kembali kode program yang sudah ada di layar TV. Kode program yang dibuat tentang Single Linked List, di bawah ini kode program yang dituliskan pada praktikum pekan 4.

2.2.1 Class Node SLL (NodeSLL_2511533028)

```

1 package pekan5_2511533028;
2
3 public class NodeSLL_2511533028 {
4     // node bagian data
5     int data_3028;
6     // Pointer ke node berikutnya
7     NodeSLL_2511533028 next_3028;
8     // Konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511533028(int data_3028)
10    {
11        this.data_3028 = data_3028;
12        this.next_3028 = null;
13    }
14
15 }

```

Gambar 2.2.1

Kode program tersebut merupakan implementasi **class node** untuk struktur data *Single Linked List* dalam bahasa Java. Pada baris awal, package `Pekan5_2511533028`; menunjukkan bahwa class ini berada dalam package tertentu. Kemudian didefinisikan class `NodeSLL_2511533028` yang berfungsi sebagai representasi satu node dalam linked list. Di dalam class ini terdapat dua atribut utama, yaitu `int data_3028` yang digunakan untuk menyimpan nilai atau data, serta `NodeSLL_2511533028 next_3028` yang berfungsi sebagai pointer atau referensi ke node berikutnya. Selanjutnya terdapat konstruktor `NodeSLL_2511533028(int data_3028)` yang digunakan untuk menginisialisasi objek node saat pertama kali dibuat, di mana nilai parameter `data_3028` akan disimpan ke dalam atribut `data_3028` menggunakan `this.data_3028 = data_3028`, dan pointer `next_3028` diatur bernilai `null` melalui `this.next_3028 = null`, yang menandakan bahwa node tersebut belum terhubung ke node lain. Secara keseluruhan, kode ini adalah dasar pembentukan elemen dalam *Single Linked List*, di mana setiap node menyimpan data dan alamat node berikutnya.

2.2.2 Class Pencarian SLL (PencarianSLL_2511533028)

```

3 public class PencarianSLL_2511533028 {
4     static boolean searchKey_3028(NodeSLL_2511533028 head_3028, int key_3028) {
5         NodeSLL_2511533028 curr = head_3028;
6         while (curr != null) {
7             if (curr.data_3028 == key_3028)
8                 return true;
9             curr = curr.next_3028;
10        }
11        return false;
12    }
13    public static void traversal(NodeSLL_2511533028 head_3028) {
14        // mulai dari head
15        NodeSLL_2511533028 curr = head_3028;
16        // telusuri sampai pointer null
17        while (curr != null) {
18            System.out.println(" "+curr.data_3028);
19            curr = curr.next_3028;
20        }
21        System.out.println();
22    }
23    public static void main(String[] args) {
24        NodeSLL_2511533028 head_3028 = new NodeSLL_2511533028(14);
25        head_3028.next_3028 = new NodeSLL_2511533028(21);
26        head_3028.next_3028.next_3028 = new NodeSLL_2511533028(13);
27        head_3028.next_3028.next_3028.next_3028 = new NodeSLL_2511533028(30);
28        head_3028.next_3028.next_3028.next_3028.next_3028 = new NodeSLL_2511533028(10);
29        System.out.print("Penelusuran SLL : ");
30        traversal(head_3028);
31        // data yang akan dicari
32        int key_3028 = 30;
33        System.out.print("cari data " +key_3028+ " = ");
34        if (searchKey_3028(head_3028, key_3028))
35            System.out.println("ketemu");
36        else
37            System.out.println("tidak ada");
38    }
39 }
40 }

```

Penelusuran SLL : 14 21 13 30 10
 cari data30 =
 Ketemu

Gambar 2.2.2

Kode program tersebut merupakan implementasi operasi **pencarian (search)** dan **penelusuran (traversal)** pada struktur data *Single Linked List* menggunakan bahasa Java. Program diawali dengan deklarasi package `Pekan5_2511533028` dan class `PencarianSLL_2511533028`. Di dalam class ini terdapat method `searchKey_2511533028` yang bertipe boolean dan berfungsi untuk mencari suatu nilai (key) dalam linked list. Method ini menerima parameter berupa `head_3028` (node awal) dan `key_3028` (data yang dicari), kemudian menggunakan variabel bantu `curr_3028` untuk menelusuri node satu per satu dengan perulangan `while (curr_3028 != null)`. Jika data pada node yang sedang diperiksa sama dengan key (`curr_3028.data_3028 == key_3028`), maka method akan mengembalikan nilai true, sedangkan jika seluruh node sudah diperiksa dan tidak ditemukan, maka akan mengembalikan false.

Selain itu, terdapat method `traversal_2511533028` yang bertujuan untuk menampilkan seluruh isi linked list. Method ini juga menggunakan variabel `curr_3028` yang dimulai dari head, lalu melakukan perulangan hingga mencapai node terakhir (null), sambil mencetak data setiap node menggunakan `System.out.print`. Setelah semua data ditampilkan, program mencetak baris baru.

Pada method main, program membuat sebuah linked list secara manual dengan membuat node pertama bernilai 14 sebagai head, kemudian menghubungkannya dengan node-node berikutnya yang berisi nilai 21, 13, 30, dan 10 melalui pointer next_3028. Setelah itu, program menampilkan isi linked list dengan memanggil method traversal. Selanjutnya ditentukan nilai yang akan dicari yaitu 30, lalu method searchKey_2511533028 dipanggil untuk mengecek apakah nilai tersebut ada di dalam linked list. Jika ditemukan, program akan menampilkan output "Ketemu", sedangkan jika tidak ditemukan akan menampilkan "tidak ada". Secara keseluruhan, kode ini menunjukkan cara membuat linked list sederhana, menampilkan isinya, serta melakukan pencarian data di dalamnya.

2.2.3 Class Tambah SLL (TambahSLL_2511533028)

```

1 public class TambahSLL_2511533028 {
2     public static NodeSLL_2511533028 insertAtFront(NodeSLL_2511533028 head_3028, int value) {
3         NodeSLL_2511533028 new_node_3028 = new NodeSLL_2511533028(value);
4         new_node_3028.next_3028 = head_3028;
5         return new_node_3028;
6     }
7     // fungsi menambahkan node di akhir SLL
8     public static NodeSLL_2511533028 insertAtEnd(NodeSLL_2511533028 head_3028, int value) {
9         // buat sebuah node dengan sebuah nilai
10        NodeSLL_2511533028 new_node_3028 = new NodeSLL_2511533028(value);
11        // jika list kosong maka node jadi head
12        if (head_3028 == null) {
13            return new_node_3028;
14        }
15        // simpan head ke variabel sementara
16        NodeSLL_2511533028 last = head_3028;
17        // selanjut ke node akhir
18        while (last.next_3028 != null) {
19            last = last.next_3028;
20        }
21        // ubah pointer
22        last.next_3028 = new_node_3028;
23        return head_3028;
24    }
25    static NodeSLL_2511533028 GetNode(int data_3028) {
26        return new NodeSLL_2511533028(data_3028);
27    }
28    static NodeSLL_2511533028 insertPos(NodeSLL_2511533028 headNode, int position, int value) {
29        NodeSLL_2511533028 head_3028 = headNode;
30        if (position < 1) {
31            System.out.print("Invalid position");
32        }
33        if (position == 1) {
34            NodeSLL_2511533028 new_node_3028 = new NodeSLL_2511533028(value);
35            new_node_3028.next_3028 = head_3028;
36            return new_node_3028;
37        }
38        else {
39            while (position-- != 0) {
40                if (position == 1) {
41                    NodeSLL_2511533028 new_node_3028 = GetNode(value);
42                    new_node_3028.next_3028 = headNode.next_3028;
43                    headNode.next_3028 = new_node_3028;
44                    break;
45                }
46                headNode = headNode.next_3028;
47            }
48            if (position != 1) {
49                System.out.print("Posisi di luar jangkauan");
50                return head_3028;
51            }
52        }
53        public static void printList(NodeSLL_2511533028 head_3028) {
54            NodeSLL_2511533028 curr = head_3028;
55            while (curr.next_3028 != null) {
56                System.out.print(curr.data_3028+"->");
57                curr = curr.next_3028;
58            }
59            if (curr.next_3028 == null) {
60                System.out.print(curr.data_3028);
61            }
62            System.out.println();
63        }
64        public static void main(String[] args) {
65            // buat linked list 2-3-5-5-6
66            NodeSLL_2511533028 head_3028 = new NodeSLL_2511533028(2);
67            head_3028.next_3028 = new NodeSLL_2511533028(3);
68            head_3028.next_3028.next_3028 = new NodeSLL_2511533028(5);
69            head_3028.next_3028.next_3028.next_3028 = new NodeSLL_2511533028(5);
70            // cetak list awal
71            System.out.print("Senarai berantai awal");
72            printList(head_3028);
73            // tambahkan node baru di depan
74            System.out.print("tambah 1 simpul di depan: ");
75            int data_3028 = 1;
76            head_3028 = insertAtFront(head_3028, data_3028);
77            // cetak update list
78            printList(head_3028);
79            // tambahkan node baru di belakang
80            System.out.print("tambah 1 simpul di belakang: ");
81            int data2_3028 = 7;
82            head_3028 = insertAtEnd(head_3028, data2_3028);
83            // cetak update list
84            printList(head_3028);
85            System.out.print("tambah 1 simpul ke data 4: ");
86            int data3_3028 = 4;
87            int pos_3028 = 4;
88            head_3028 = insertPos(head_3028, pos_3028, data3_3028);
89            // cetak update list

```

```

89     printList(head_3028);
90
91 }
92
93 }

```

Senarai berantai awal : 2-->3-->5-->6
 tambah 1 simpul di depan : 1-->2-->3-->5-->6
 Tambah 1 simpul di belakang : 1-->2-->3-->5-->6-->7
 tambah 1 simpul ke data 4 : 1-->2-->3-->4-->5-->6-->7

Gambar 2.2.3

Kode program tersebut merupakan implementasi operasi **penambahan (insert)** pada struktur data *Single Linked List* dalam bahasa Java. Program diawali dengan deklarasi package `Pekan5_2511533028` dan class `TambahSLL_2511533028` yang berisi beberapa method untuk menambahkan node pada berbagai posisi, yaitu di depan, di akhir, dan pada posisi tertentu, serta method untuk mencetak isi linked list.

Method `insertAtFront_2511533028` digunakan untuk menambahkan node di bagian depan (head). Cara kerjanya adalah dengan membuat node baru menggunakan nilai yang diberikan, kemudian pointer next dari node baru diarahkan ke head lama, dan node baru tersebut dikembalikan sebagai head yang baru. Selanjutnya, method `insertAtEnd_2511533028` berfungsi untuk menambahkan node di bagian akhir linked list. Jika linked list masih kosong (`head == null`), maka node baru langsung dijadikan head. Jika tidak kosong, program akan menelusuri hingga node terakhir menggunakan perulangan `while`, kemudian menghubungkan node terakhir tersebut dengan node baru melalui pointer `next`.

Method `GetNode_2511533028` merupakan fungsi sederhana untuk membuat node baru berdasarkan data yang diberikan. Kemudian method `insertPos` digunakan untuk menambahkan node pada posisi tertentu dalam linked list. Jika posisi bernilai 1, maka proses sama seperti insert di depan. Jika posisi lebih dari 1, maka program akan menelusuri node hingga posisi yang diinginkan, lalu menyisipkan node baru dengan cara mengatur pointer agar node baru berada di antara node sebelumnya dan node berikutnya. Jika posisi tidak valid atau di luar jangkauan, program akan menampilkan pesan kesalahan.

Selain itu, terdapat method `printList_2511533028` yang berfungsi untuk menampilkan seluruh isi linked list. Method ini menelusuri setiap node dari head hingga node terakhir, lalu mencetak data dengan format berantai (menggunakan

tanda panah “->”), dan diakhiri dengan baris baru.

Pada method main, program membuat linked list awal secara manual dengan isi data $2 \rightarrow 3 \rightarrow 5 \rightarrow 6$. Kemudian program menampilkan list awal, lalu melakukan beberapa operasi penambahan, yaitu menambahkan node dengan nilai 1 di depan, menambahkan node dengan nilai 7 di akhir, dan menambahkan node dengan nilai 4 pada posisi ke-4. Setelah setiap operasi, linked list dicetak kembali untuk menunjukkan hasil perubahan. Secara keseluruhan, kode ini menggambarkan bagaimana melakukan manipulasi data pada Single Linked List, khususnya operasi penambahan node di berbagai posisi serta menampilkan hasilnya.

2.2.4 Class Reverse Date (ReverseDate_2511533028)

```

1 public class ReverseDate_2511533028 {
2     // fungsi untuk menghapus head
3     public static NodeSLL_2511533028 deleteHead(NodeSLL_2511533028 head_3028) {
4         // jika list kosong
5         if (head_3028 == null)
6             return null;
7         // simpan head ke node berikutnya
8         head_3028 = head_3028.next_3028;
9         // return head baru
10        return head_3028;
11    }
12    // fungsi untuk menghapus node terakhir SLL
13    public static NodeSLL_2511533028 deleteLastNode(NodeSLL_2511533028 head_3028) {
14        // jika list kosong, return null
15        if (head_3028 == null) {
16            return null;
17        }
18        // jika list satu node, hapus node dan return null
19        if (head_3028.next_3028 == null) {
20            return null;
21        }
22        // kemudia node terakhir ke dua
23        NodeSLL_2511533028 secondLast = head_3028;
24        while (secondLast.next_3028.next_3028 != null) {
25            secondLast = secondLast.next_3028;
26        }
27        // hapus node terakhir
28        secondLast.next_3028 = null;
29        return head_3028;
30    }
31    // fungsi untuk menghapus node di posisi tertentu
32    public static NodeSLL_2511533028 deleteNode(NodeSLL_2511533028 head_3028, int position) {
33        NodeSLL_2511533028 temp_3028 = head_3028;
34        NodeSLL_2511533028 prev_3028 = null;
35        // jika linked list null
36        if (temp_3028 == null)
37            return head_3028;
38        // hapus jika head dihapus
39        if (position == 1) {
40            head_3028 = temp_3028.next_3028;
41            return head_3028;
42        }
43        // simpan node yang akan dihapus
44        // temp_3028 ke node yang di hapus
45        for (int i = 1; temp_3028 != null && i < position; i++) {
46            prev_3028 = temp_3028;
47            temp_3028 = temp_3028.next_3028;
48        }
49        // jika ditemukan, hapus node
50        if (temp_3028 != null) {
51            prev_3028.next_3028 = temp_3028.next_3028;
52        } else {
53            System.out.println("Data tidak ada");
54        }
55        return head_3028;
56    }
57    // fungsi untuk cetak SLL
58    public static void printList(NodeSLL_2511533028 head_3028) {
59        NodeSLL_2511533028 curr = head_3028;
60        while (curr.next_3028 != null) {
61            System.out.print(curr.data_3028 + " -> ");
62            curr = curr.next_3028;
63        }
64        if (curr.next_3028 == null) {
65            System.out.print(curr.data_3028);
66        }
67        System.out.println();
68    }
69    // fungsi main
70    public static void main(String[] args) {
71        // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
72        NodeSLL_2511533028 head_3028 = new NodeSLL_2511533028(1);
73        head_3028.next_3028 = new NodeSLL_2511533028(2);
74        head_3028.next_3028.next_3028 = new NodeSLL_2511533028(3);
75        head_3028.next_3028.next_3028.next_3028 = new NodeSLL_2511533028(4);
76        head_3028.next_3028.next_3028.next_3028.next_3028 = new NodeSLL_2511533028(5);
77        head_3028.next_3028.next_3028.next_3028.next_3028.next_3028 = new NodeSLL_2511533028(6);
78        // cetak list awal
79        System.out.println("list awal:");
80        printList(head_3028);
81        // hapus head
82        head_3028 = deleteHead(head_3028);
83        System.out.println("list setelah head dihapus:");
84        printList(head_3028);
85        // hapus node terakhir
86        head_3028 = deleteLastNode(head_3028);
87        System.out.println("list setelah simpan terakhir di hapus:");
88        printList(head_3028);
89        // Hapus node di position 2
90        head_3028 = deleteNode(head_3028, position);
91        // Print list after deletion
92        System.out.println("list setelah posisi 2 dihapus:");
93        printList(head_3028);
94    }
95 }

```

```

List awal :
1-->2-->3-->4-->5-->6
List setelah head dihapus :
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus :
2-->3-->4-->5-->6
List setelah posisi 2 dihapus :
2-->4-->5-->6

```

Gambar 2.2.4

Kode program tersebut merupakan implementasi operasi **penghapusan (delete)** pada struktur data *Single Linked List* menggunakan bahasa Java. Program berada dalam package `Pekan5_2511533028` dan memiliki class `HapusSLL_2511533028` yang berisi beberapa method untuk menghapus node dari berbagai posisi, yaitu di awal (head), di akhir, dan pada posisi tertentu, serta method untuk menampilkan isi linked list.

Method `deleteHead_2511533028` berfungsi untuk menghapus node pertama (head). Jika linked list kosong (`head == null`), maka method langsung mengembalikan null. Jika tidak, head akan dipindahkan ke node berikutnya (`head.next`), sehingga node pertama sebelumnya otomatis terhapus, lalu mengembalikan head yang baru. Selanjutnya, method `removeLastNode_2511533028` digunakan untuk menghapus node terakhir. Jika linked list kosong atau hanya memiliki satu node, maka hasilnya adalah null. Jika lebih dari satu node, program menggunakan variabel `secondLast` untuk menelusuri hingga node sebelum terakhir, kemudian memutus hubungan node terakhir dengan mengatur `secondLast.next = null`.

Method `deleteNode_2511533028` digunakan untuk menghapus node pada posisi tertentu. Program menggunakan dua variabel bantu, yaitu `temp` sebagai penunjuk node yang sedang diperiksa dan `prev` sebagai node sebelumnya. Jika posisi yang dihapus adalah posisi pertama, maka sama seperti menghapus head. Jika bukan, program melakukan perulangan untuk mencapai posisi yang diinginkan. Jika node ditemukan, maka pointer dari node sebelumnya (`prev.next`) akan diarahkan ke node setelah node yang dihapus, sehingga node tersebut terlepas dari linked list. Jika posisi tidak ditemukan, program akan menampilkan pesan "Data tidak ada".

Method `printList_2511533028` digunakan untuk mencetak isi linked list. Program menelusuri setiap node dari head hingga node terakhir, mencetak data

dengan format berantai menggunakan tanda panah (-->), lalu menampilkan node terakhir tanpa panah dan diakhiri dengan baris baru.

Pada method main, program membuat linked list secara manual dengan isi data $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow \text{null}$. Kemudian program menampilkan list awal, dilanjutkan dengan operasi penghapusan secara bertahap, yaitu menghapus node pertama (head), menghapus node terakhir, dan menghapus node pada posisi ke-2. Setelah setiap proses penghapusan, linked list ditampilkan kembali untuk menunjukkan perubahan yang terjadi. Secara keseluruhan, kode ini menjelaskan bagaimana melakukan berbagai operasi penghapusan pada Single Linked List serta menampilkan hasilnya setelah setiap operasi dilakukan.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan seluruh kode program yang telah dibahas (pembuatan node, pencarian, penambahan, dan penghapusan), dapat disimpulkan bahwa *Single Linked List* merupakan struktur data dinamis yang tersusun dari node-node yang saling terhubung melalui pointer. Implementasi program menunjukkan bagaimana data dapat dikelola secara fleksibel melalui berbagai operasi utama, yaitu traversal (penelusuran), search (pencarian), insert (penambahan), dan delete (penghapusan). Setiap operasi memiliki mekanisme yang berbeda namun tetap berpusat pada manipulasi pointer antar node. Penambahan dapat dilakukan di depan, belakang, maupun posisi tertentu dengan cara mengatur hubungan node baru ke node lain, sedangkan penghapusan dilakukan dengan memutus hubungan node yang ingin dihapus dari rangkaian linked list. Pencarian dan traversal dilakukan secara berurutan dari head hingga node terakhir karena tidak adanya akses langsung seperti pada array.

Secara keseluruhan, kode-kode tersebut menggambarkan bahwa pengelolaan Single Linked List sangat bergantung pada ketelitian dalam mengatur pointer agar struktur data tetap konsisten. Pemahaman ini menjadi dasar penting dalam mempelajari struktur data lanjutan serta meningkatkan kemampuan logika dalam pemrograman.

3.2 Saran

Adapun saran yang dapat diberikan berdasarkan materi ini adalah sebagai berikut:

1. Pastikan setiap perubahan pada linked list (insert/delete) dilakukan dengan benar agar tidak terjadi kehilangan data atau error (misalnya null pointer).
2. Sebaiknya tambahkan pengecekan posisi atau kondisi list agar program lebih aman, seperti validasi posisi saat insert atau delete.
3. Pisahkan fungsi-fungsi ke dalam class yang berbeda (misalnya class `LinkedList` dan `Node`) agar struktur program lebih terorganisir.
4. Kembangkan program dengan fitur tambahan seperti menghitung jumlah

node, mencari nilai maksimum/minimum, atau reverse linked list.

5. Cobalah berbagai skenario (list kosong, satu node, banyak node) untuk memahami cara kerja linked list secara lebih mendalam.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson dan R. L. Rivest, “Introducing to Algorithms,” 2009. [Online].
- [2] M. T. Goodrich, R. Tamassia dan M. H. Goldwasser, “Data Structures and Algorithms in Java,” 2014. [Online].

LAPORAN TUGAS PRAKTIKUM

SOAL:

1. Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out). **Class Pasien (Pasien_2511533028)**

```
1 package tugaspekan5_2511533028;
2
3 public class Pasien_2511533028 {
4     String namaPasien_3028;
5     String penyakit_3028;
6     int nomorAntrian_3028;
7     Pasien_2511533028 next_3028;
8
9     // Constructor
10    public Pasien_2511533028(String nama, String penyakit, int nomor) {
11        this.namaPasien_3028 = nama;
12        this.penakit_3028 = penyakit;
13        this.nomorAntrian_3028 = nomor;
14        this.next_3028 = null;
15    }
16
17    // Getter
18    public String getNama_3028() {
19        return namaPasien_3028;
20    }
21
22    public String getPenyakit_3028() {
23        return penyakit_3028;
24    }
25
26    public int getNomor_3028() {
27        return nomorAntrian_3028;
28    }
29
30    public Pasien_2511533028 getNext_3028() {
31        return next_3028;
32    }
33
34    // Setter
35    public void setNext_3028(Pasien_2511533028 next) {
36        this.next_3028 = next;
37    }
38
39 }
```

Gambar 1

Kode program ini merupakan implementasi **Queue (Antrian) Melingkar (Circular Queue)** dalam bahasa Java yang digunakan untuk simulasi antrian loket, terdiri dari satu kelas bernama AntrianLoket_2511533028 dalam package Pekan4_2511533028.

Pada bagian awal kelas, dideklarasikan beberapa atribut yaitu front_3028 dan rear_3028 sebagai penanda posisi depan dan belakang antrian, size_3028 untuk ukuran aktual antrian, max_3028 untuk kapasitas maksimum, serta array queue_3028[] bertipe String sebagai tempat penyimpanan nama pelanggan.

Konstruktor kelas menerima parameter `max_3028` untuk menentukan kapasitas antrian, kemudian menginisialisasi `front` dan `rear` dengan nilai `-1` sebagai tanda bahwa antrian masih kosong, sekaligus mengalokasikan array sesuai kapasitas yang diberikan.

Terdapat dua method pengecekan kondisi antrian, yaitu `isFull_3028` yang mengembalikan nilai `true` apabila posisi `rear` telah mencapai batas maksimum kapasitas, dan `isEmpty_3028` yang mengembalikan `true` apabila nilai `front` masih bernilai `-1` yang berarti belum ada data dalam antrian. Method `enqueue_3028` berfungsi untuk menambahkan nama pelanggan baru ke dalam antrian. Jika antrian sudah penuh maka proses dibatalkan, jika antrian masih kosong maka `front` diset ke `0`, kemudian posisi `rear` digeser menggunakan operasi modulo agar antrian bersifat melingkar, lalu nama pelanggan disimpan pada posisi tersebut. Method `dequeue_3028` berfungsi untuk mengambil dan menghapus pelanggan dari posisi terdepan antrian. Apabila antrian kosong maka dikembalikan nilai `null`, apabila hanya tersisa satu elemen maka `front` dan `rear` direset ke `-1`, dan apabila masih terdapat elemen lain maka `front` digeser maju secara melingkar menggunakan operasi modulo.

Method `display_3028` digunakan untuk menampilkan seluruh isi antrian dari posisi depan hingga belakang secara berurutan dengan nomor urut, dan apabila antrian kosong akan ditampilkan pesan "Antrian Kosong". Method `front_3028` digunakan untuk melihat data pelanggan yang berada di posisi terdepan tanpa menghapusnya dari antrian, dan mengembalikan `null` apabila antrian kosong. Terakhir, method `reverse_3028` berfungsi untuk membalik seluruh urutan antrian dengan cara mengeluarkan semua elemen ke dalam array sementara temp menggunakan `dequeue`, kemudian memasukkannya kembali ke antrian dalam urutan terbalik menggunakan `enqueue`, sehingga pelanggan yang semula berada di posisi paling belakang akan menjadi yang paling depan.


```

122●      case 3:
123          rs.tampilkanAntrian();
124          break;
125
126●      case 4:
127          System.out.print("Masukkan nama: ");
128          String cari = input.nextLine();
129          rs.cariPasien(cari);
130          break;
131
132●      case 5:
133          rs.cekStatus();
134          break;
135
136●      case 6:
137          System.out.println("Keluar...");
138          break;
139
140●      default:
141          System.out.println("Pilihan tidak valid!");
142      }
143  } while (pilihan != 6);
144
145      input.close();
146  }

```

Gambar 2

Program **RumahSakit_2511533028** merupakan program berbasis Java yang dibuat untuk mensimulasikan sistem antrian pasien di rumah sakit menggunakan struktur data **Singly Linked List**. Program ini memungkinkan pengguna untuk melakukan berbagai operasi seperti mendaftarkan pasien baru, memanggil pasien sesuai urutan antrian, menampilkan daftar pasien yang sedang menunggu, mencari data pasien berdasarkan nama, serta melihat status antrian. Konsep yang digunakan dalam program ini adalah **FIFO (First In First Out)**, yaitu pasien yang datang terlebih dahulu akan dilayani terlebih dahulu.

Pada bagian awal program terdapat deklarasi package dan import library `Scanner` yang digunakan untuk menerima input dari pengguna melalui keyboard. Selanjutnya dibuat class utama bernama `RumahSakit_2511533028` yang berfungsi sebagai pusat pengelolaan data antrian pasien. Di dalam class ini terdapat variabel `head_3028` yang berperan sebagai penunjuk node pertama pada linked list serta variabel `counter_3028` yang digunakan untuk memberikan nomor antrian secara otomatis kepada setiap pasien yang mendaftar.

Method `daftarPasien()` digunakan untuk menambahkan pasien baru ke dalam antrian. Ketika pasien mendaftar, nomor antrian akan bertambah secara otomatis melalui variabel `counter`. Setelah itu dibuat objek pasien baru yang berisi nama pasien, keluhan atau penyakit, dan nomor antrian. Jika antrian masih kosong, maka pasien tersebut langsung menjadi node pertama atau head. Namun

jika antrian sudah berisi pasien lain, program akan melakukan traversal hingga mencapai node terakhir, kemudian menambahkan pasien baru pada bagian akhir linked list. Setelah proses berhasil, sistem akan menampilkan pesan bahwa pasien telah berhasil didaftarkan beserta nomor antriannya.

Method `panggilPasien()` berfungsi untuk melayani pasien yang berada di posisi paling depan antrian. Jika antrian kosong, program akan menampilkan pesan bahwa tidak ada pasien yang dapat dipanggil. Sebaliknya, jika antrian berisi data pasien, maka informasi pasien terdepan akan ditampilkan, meliputi nama dan keluhan yang dialami. Setelah pasien dipanggil, node pertama akan dihapus dengan cara memindahkan posisi head ke node berikutnya. Dengan demikian pasien yang sebelumnya berada pada urutan kedua akan menjadi pasien terdepan dalam antrian.

Untuk melihat seluruh isi antrian, program menyediakan method `tampilkanAntrian()`. Method ini akan melakukan traversal mulai dari node pertama hingga node terakhir. Setiap data pasien yang ditemukan akan ditampilkan ke layar, meliputi nomor antrian, nama pasien, dan keluhan yang diderita. Jika antrian tidak memiliki data, maka program akan memberikan informasi bahwa antrian masih kosong.

Program juga memiliki fitur pencarian pasien melalui method `cariPasien()`. Method ini menerima parameter berupa nama pasien yang ingin dicari. Proses pencarian dilakukan dengan menelusuri setiap node pada linked list dan membandingkan nama yang tersimpan dengan nama yang dimasukkan pengguna menggunakan fungsi `equalsIgnoreCase()`, sehingga perbedaan huruf besar dan kecil tidak mempengaruhi hasil pencarian. Jika data pasien ditemukan, maka informasi lengkap pasien akan ditampilkan. Namun jika tidak ditemukan hingga akhir linked list, program akan menampilkan pesan bahwa pasien tidak ditemukan.

Selain itu, terdapat method `cekStatus()` yang digunakan untuk mengetahui kondisi antrian saat ini. Method ini menghitung jumlah seluruh pasien yang sedang menunggu dengan melakukan traversal pada linked list dan

menghitung jumlah node yang ada. Setelah itu program akan menampilkan jumlah pasien dalam antrian serta nama pasien yang berada di posisi paling depan. Informasi ini berguna untuk mengetahui tingkat kepadatan antrian dan pasien yang akan segera dilayani.

Pada method `main()`, program menampilkan menu utama yang berisi enam pilihan, yaitu daftar pasien, panggil pasien, tampilkan antrian, cari pasien, cek status, dan keluar dari program. Menu ini dijalankan menggunakan perulangan `do-while`, sehingga pengguna dapat terus menggunakan program sampai memilih opsi keluar. Setiap pilihan menu akan memanggil method yang sesuai sehingga seluruh fitur antrian dapat digunakan secara interaktif.

Secara keseluruhan, program **RumahSakit_2511533028** merupakan implementasi sederhana sistem antrian rumah sakit yang memanfaatkan struktur data **Singly Linked List**. Penggunaan linked list memungkinkan proses penambahan dan penghapusan data dilakukan secara dinamis tanpa harus menentukan kapasitas penyimpanan di awal. Dengan adanya fitur pendaftaran, pemanggilan, pencarian, penampilan data, dan pengecekan status antrian, program ini telah mampu menggambarkan mekanisme dasar pengelolaan antrian pasien pada sebuah rumah sakit secara efektif dan terstruktur.

OUTPUT UNTUK MENAMBAH :

```
=== Antrian Rumah Sakit NIM: 2511533028 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 1
Nama: Atha
Keluhan: Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1
```

```

=== Antrian Rumah Sakit NIM: 2511533028 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 1
Nama: Ary
Keluhan: Flu
Pasien berhasil didaftarkan! Nomor Antrian: 2

```

```

=== Antrian Rumah Sakit NIM: 2511533028 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 1
Nama: Denis
Keluhan: Batuk
Pasien berhasil didaftarkan! Nomor Antrian: 3

```

OUTPUT UNTUK PANGGIL PASIEN :

```

=== Antrian Rumah Sakit NIM: 2511533028 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 2
Memanggil Pasien:
Nama: Atha
Keluhan: Demam

```

OUTPUT UNTUK TAMPILKAN ANTRIAN :

```

=== Antrian Rumah Sakit NIM: 2511533028 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 3
No: 2 | Nama: Ary | Keluhan: Flu
No: 3 | Nama: Denis | Keluhan: Batuk

```

OUTPUT UNTUK CARI PASIEN :

```

=== Antrian Rumah Sakit NIM: 2511533028 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 4
Masukkan nama: Ary
Pasien ditemukan!
No: 2
Nama: Ary
Keluhan: Flu

```

OUTPUT UNTUK CEK STATUS :

```

=== Antrian Rumah Sakit NIM: 2511533028 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 5
Jumlah Pasien: 2
Pasien terdepan: Ary

```

OUTPUT UNTUK KELUAR :

```

=== Antrian Rumah Sakit NIM: 2511533028 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 6
Keluar...

```